

CS 535: Assignment 03

Programming Assignments (90%)

- **This assignment is in C++.**
- Copy `src/app/Assign02.cpp` and name it **`src/app/Assign03.cpp`**
- Change the application name from "Assign02" to "Assign03"
- Change the window title to be "Assign03: " + your SITNET ID
 - o E.g., "Assign03: realemj"
- Make a copy of the `vulkanshaders/Assign02` folder and name it **`vulkanshaders/Assign03`**
- Modify **`CMakeLists.txt`** by adding the following line to the end of the file:
 - o `CREATE_VULKAN_EXECUTABLE(Assign03)`
- Make sure the program configures, compiles, and runs as-is

Assign03.cpp

- Add the following includes:
 - o **`#include <assimp/Importer.hpp>`**
 - o **`#include <assimp/scene.h>`**
 - o **`#include <assimp/postprocess.h>`**
- Add a **`glm::vec3`** normal to **`ProVertex`**
- Create a struct to hold vertex shader push constants: **`UPushVertex`**
 - o Add one field for the model matrix:
`alignas(16) glm::mat4 modelMat`
 - o Add one field to determine how much of color is from normals:
`alignas(16) float useNormalAsColor;`
- Add the following globals:
 - o Extra Z rotation: **`float rotAngleZ = 0.0f;`**
 - o Weight for using normals as the color: **`float useNormalAsColor = 1.0f;`**
 - o Extra Z translation: **`float zOffset = 0.5f;`**
 - o Flip winding order on load (since Z is actually flipped and following left-hand rule):
`bool flipWindingOrder = true;`

- Add the following keys to the **key_callback()** function:
 - If the action is either GLFW_PRESS or GLFW_REPEAT, check for the following keys:
 - GLFW_KEY_J
 - Add 1.0 to rotAngleZ
 - GLFW_KEY_K
 - Subtract 1.0 from rotAngleZ
 - GLFW_KEY_O
 - Add 0.1 to useNormalAsColor BUT clamp to [0,1]
 - GLFW_KEY_I (letter i)
 - Subtract 0.1 from useNormalAsColor BUT clamp to [0,1]
 - GLFW_KEY_U
 - Add 0.05 to zOffset
 - GLFW_KEY_Y
 - Subtract 0.05 from zOffset
- Add a new function: **glm::mat4 assimpToGLM(aiMatrix4x4 &a)**
 - Convert from row-major aiMatrix4x4 to column-major glm::mat4
- Add the following function for generating a transformation to rotate around the LOCAL Z axis: **glm::mat4 makeRotateZ(glm::vec3 offset)**
 - Generate transformation matrices (with glm) and form a composite transformation to perform the following IN ORDER:
 - Translate by NEGATIVE offset
 - Rotate rotAngleZ around the Z axis
 - **REMEMBER TO CONVERT rotAngleZ to RADIANS!!!!**
 - Translate by offset
 - Return the composite transformation
- Create a new function:

void renderSceneNode(vk::CommandBuffer &commandBuffer, pro::VulkanPipelineData &pipelineData, vector<pro::VulkanMesh> &allMeshes, aiNode *node, glm::mat4 parentMat, int level)

 - Get the transformation for the current node, which is an aiMatrix4x4:
node->mTransformation
 - Convert the transformation to a **glm::mat4 nodeT** using the aiMatToGLM4() function
 - Compute the current model matrix: **glm::mat4 modelMat = parentMat*nodeT**
 - Get location of current node by:
 - Grabbing the last column of modelMat, which is a vec4
 - Remember that glm matrices are stored in column-major format, so modelMat[3] gives you the last column.

- Convert this vec4 to a vec3 pos
 - Call makeRotateZ(pos) to get a proper local Z rotation: ***R***
 - Add in the Z translation based on zOffset: ***R = glm::translate(glm::vec3(0,0,zOffset))*R;***
 - Generate a temporary model matrix model matrix as:
 - ***glm::mat4 tmpModel = R * modelMat***
 - Create an instance of **UPushVertex** and store ***tmpModel*** as the model matrix as well as the current value of useNormalAsColor.
 - Use **commandBuffer.pushConstants()** to push up the UPushVertex data
 - For each mesh in the NODE (***node->mNumMeshes*** meshes total)
 - Get the index of the mesh: ***int index = node->mMeshes[i]***
 - Call **recordDrawVulkanMesh()** on each mesh
allMeshes.at[index]
 - Call renderSceneNode() on each child of the NODE (***node->mNumChildren*** children total)
 - Command buffer, pipeline data, all meshes: same as passed in
 - Node: ***node->mChildren[i]***
 - Parent matrix: ***modelMat*** (NOT ***tmpModel***!)
 - Level: ***level + 1***
- Change **recordFrame()**:
 - Add **const aiScene *scene** as a parameter.
 - Instead of looping through and drawing all meshes, call renderSceneNode() ONCE:
 - Node: ***scene->mRootNode***
 - Parent matrix: ***glm::mat4(1.0)***
 - Level: ***0***
- Create a new function: **pro::HostMesh<ProVertex> extractMeshData(aiMesh *mesh)**
 - The overall purpose of this function to grab the vertex positions and shape indices from the given aiMesh and store this information in a HostMesh struct.
 - Create a new pro::HostMesh<ProVertex> to hold the information.
 - Loop through all vertices in the aiMesh (mesh->mNumVertices):
 - Create a ProVertex.
 - Grab the vertex position information from mesh->mVertices[i] and store it in the ProVertex's position.
 - Note: mVertices is an array of aiVector3D structs, so you will have to somehow convert to glm::vec3 structs
 - Set the color of the ProVertex to **any color other than (0,0,0,1) or the background.**
 - You may use different colors per vertex using any reasonable scheme.
 - However, alpha values should always be 1.0f.
 - Also grab and store the vertex normals.

- Add the ProVertex to the HostMesh's vertices list.
 - Loop through all faces in the aiMesh (mesh->mNumFaces):
 - Grab the aiFace *face* from mesh->mFaces[i].
 - Loop through the number of indices for this face (face.mNumIndices):
 - Add each index for the face (face.mIndices[k]) to the HostMesh's list of indices.
 - Return the HostMesh.
- In the main function:
 - BEFORE any of the other code in main:
 - The model to load will be provided on the command line:
 - Use "sampleModels/sphere.obj" as your default model path.
 - If argc >= 2, grab argv[1] as the model path to load and convert to a string.
 - **NOTE: Do NOT use cin!! You MUST use the command line arguments!**
 - Load the model (given by the model path) using Assimp to get an aiScene
 - Use the following options (OR'ed together) and store them in **unsigned int aiFlags**:
 - aiProcess_Triangulate
 - aiProcess_FlipUVs
 - aiProcess_GenNormals
 - aiProcess_JoinIdenticalVertices
 - If flipWindingOrder is true → also OR **aiProcess_FlipWindingOrder**
 - Create the importer and load the model with the correct filename and flags.
 - Check to make sure the model loaded correctly
 - Print error and exit if ANY of these are true:
 - The scene object is null
 - scene->mFlags & AI_SCENE_FLAGS_INCOMPLETE
 - scene->mRootNode is null
 - BEFORE the render loop, in the area where the host and device meshes are created:
 - Add the normal to the **pipelineCreateInfo.attribDesc** list.
 - Comment out existing host mesh and VulkanMesh creation code.
 - Make sure you have the following lists:
 - A vector of **pro::HostMesh<ProVertex>**'s for all host mesh data
 - A vector of **pro::VulkanMesh**'s for all meshes
 - Pre-allocate space for both lists using **resize()** based on the number of meshes in the Assimp scene (mNumMeshes):
 - **Example: allHostMeshes.resize(scene->mNumMeshes);**
 - For each **mesh** in the Assimp scene (mMeshes with mNumMeshes):

- Call **extractMeshData** to get a **HostMesh** from each scene->mMeshes[i]
- Call **pro::createVulkanMesh()** to get a HOST-VISIBLE **VulkanMesh**
- Use **pro::copyToHostVisibleVulkanMesh()** to copy the host mesh into the VulkanMesh
- **IN THE RENDER LOOP:**
 - When calling **recordFrame()**, remember to also pass in the loaded scene object.

shader.vert

- Add input for normal data.
- Add the appropriate push constant struct.
- For gl_Position, multiply pc.modelMat by the input vertex position (IN THAT ORDER).
- Compute the world_normal with the correct normal transform version of pc.modelMat
- Get the "color_normal" by rescaling world_normal from [-1,1] to [0,1] and making it a vec4 with alpha 1.0f.
- Change interColor to be a mix of color and color_normal based on pc.useNormalAsColor:
interColor = mix(color, color_normal, pc.useNormalAsColor);

Screenshots (5%)

Sample models have been provided for this assignment. To pass in command line arguments:

- Go to .vscode/launch.json
- Change "args" entry to ["./sampleModels/teapot.obj"],

To test NO command line arguments:

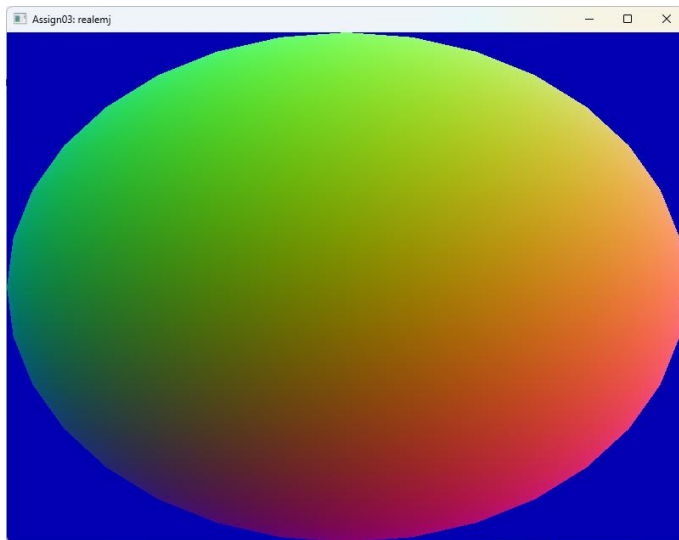
- Change "args" entry to []

Remember: your program should load "./sampleModels/sphere.obj" by default if no command line arguments are provided.

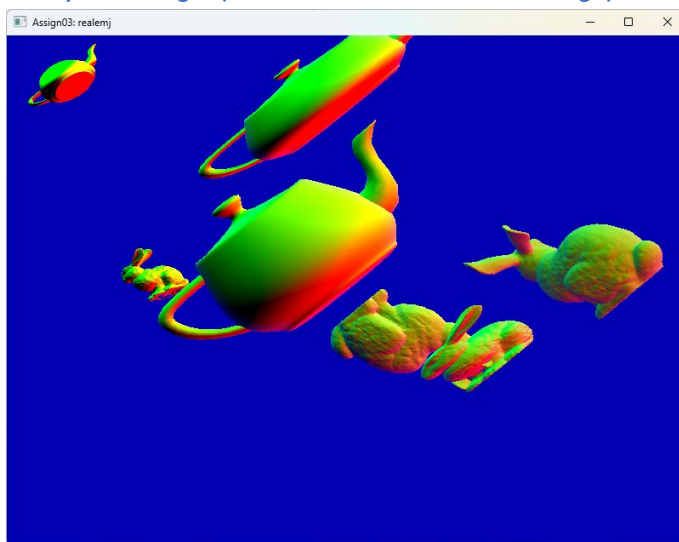
For this part of the assignment, **upload TWO screenshots** of the application window (note that window title should be "Assign03: yoursitnetid"):

- **One displaying sphere.obj after moving the sphere back so it is visible → Assign03_sphere.png**
- **One displaying bunnyteatime.glb after J rotation → Assign03_teatime_J.png**

sphere.obj (color need not match this image):



Bunnyteatime.glb (color need not match this image):



Copy these images into the **screenshots** folder.

README (5%)

- Under the "Applications" section, add a subsection (same format as Assign01) very briefly describing your Assign03 code.

Grading

Your OVERALL assignment grade is weighted as follows:

- 90% - Programming
- 5% - Screenshots
- 5% - README

I reserve the right to take points off for not meeting the specifications in this assignment description.

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	Maximum Penalty
Code that does not compile	60
Incorrect renderSceneNode()	20
Incorrect vertex shader	15
Incorrect extractMeshData()	15
Incorrect model loading (including default filename, flags, and error-checking)	15
Incorrect mesh creation and copying	15
Keyboard functions incorrect	5
Incorrect makeRotateZ	5
Technically correct code, but poor code design	5
Missing assignment Git branch	5
Missing screenshots	5
README update missing/incorrect	5

In general, these are things that will be penalized:

- **Code that does not compile (up to 60 points off!)**
- Sloppy or poor coding style
- Bad coding design principles
- Code that crashes, does not run, or takes a VERY long time to complete
- Using code from ANY source other than the course materials
- Collaboration on code of ANY kind; this is an INDIVIDUAL PROJECT
- Sharing code with other people in this class or using code from this or any other related class
- Output that is incorrect
- Algorithms/implementations that are incorrect
- Submitting improper files
- Failing to submit ALL required files